LCI workshop
on high performance clustered computing

LCI Advanced Workshop 2025: **Job Arrays**

Willy Dizon
Senior Systems Analyst
Arizona State University
willy.dizon@asu.edu

ASU Research Computing

Job Array Overview

- Introduction and Objectives
  - What is the goal?
  - Who will use these?
  - Why use Job Arrays vs individual jobs
- Fundamentals Recap
  - Syntax
  - Examples w/ Environment Variables
- Control Mechanisms
  - MaxArraySize
  - Indexing
  - Throttling
  - Insufficient MaxJobCount

What is the Goal?

Things Users Care About:

- Efficient parallelization
- Batch jobs into easily-trackable units
- Simplified submission / resubmission

Things Admins Care About:

- Job count minimization
- Scheduler loop efficiency
- Reduced user handholding

Who Uses Job Arrays?

Job arrays should be used by all users who submit hundreds or thousands of jobs that all require roughly the same amount of compute resources.

One tell-tale sign of a user who could benefit from job arrays is that they copy their batch script repeatedly, rename it, and point it at different files.

Why Use Job Arrays?

Why should Admins Care?

- Scheduler loop only requires 1 evaluation, not N evaluations:
  This makes for a more responsive scheduler, requires less memory, and permits spending more time evaluating backfills, etc.
- Quicker scheduling loops means fewer scheduler timeouts:
  Lessens chance of job starvations (idle, unused resources for longer periods)
- Easier management of users' jobs
  Jobs are logically connected, so holding/cancelling/requeuing is very straightforward

Why Should Users Care?

- Job scheduling needs to happen **once**:
  Rather than have N jobs all having their own priority, once the job array is scheduled, all the subjobs share that single priority.
- Easier management of their jobs
  Jobs are logically connected, so holding/cancelling/requeuing is very straightforward

Why Use Job Arrays?

What does this mean, practically?

Job scheduling needs to happen **once**.

```
[root@lci-head-02-1 ~]# sbatch --array=1-50 noop.sh
Submitted batch job 11
[root@lci-head-02-1 ~]# squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
         11_[9-50]   general  noop.sh     root PD       0:00      1 (Resources)
              11_5   general  noop.sh     root  R       0:00      1 lci-compute-02-1
              11_6   general  noop.sh     root  R       0:00      1 lci-compute-02-1
              11_7   general  noop.sh     root  R       0:00      1 lci-compute-02-2
              11_8   general  noop.sh     root  R       0:00      1 lci-compute-02-2
[root@lci-head-02-1 ~]# scontrol show job | grep Priority
   Priority=50000000 Nice=0 Account=root QOS=normal
   Priority=50000000 Nice=0 Account=root QOS=normal
   Priority=50000000 Nice=0 Account=root QOS=normal
   Priority=50000000 Nice=0 Account=root QOS=normal
```

Have your users ask themselves: would they rather have N run with the same shared priority? Or would they prefer all their queued jobs run at different times, and their later jobs all suffering a poor priority score?

Fundamentals Recap

Easily Start multiple jobs with varying inputs.

```
$ sbatch --array=1,3,5 noop.sh
Submitted batch job 34396612
$ squeue -u $USER
     JobID  PRIORITY  PARTITION/QOS  NAME       STATE  TIME  TIME_LIMIT  Node/Core/GPU
NODELIST(REASON)
34396612_1  99977884     htc/public  noop.sh  RUNNING  0:02        1:00  2/2/NA         sc[099-100]
34396612_3  99977884     htc/public  noop.sh  RUNNING  0:02        1:00  2/2/NA         sc[091-092]
34396612_5  99977884     htc/public  noop.sh  RUNNING  0:02        1:00  2/2/NA         sc[092-093]
$ scancel 34396612_5
```

```
34396612_5
^^^^^^^^   Job ID for whole Array:   $SLURM_ARRAY_JOB_ID
        ^ Array task ID for job:    $SLURM_ARRAY_TASK_ID
```

`--array takes comma-separated integers, or ranges, e.g., 1-31; this becomes the suffix to the job ID`

Best examples: https://slurm.schedmd.com/job_array.html

Job Array - A Toy Example

```
$ cat noop.sh
#!/bin/bash
# Grab the line matching this task's array index (1-based):
ITEM="$(sed -n "${SLURM_ARRAY_TASK_ID}p" samples.txt)"
echo "task_id=${SLURM_ARRAY_TASK_ID} item=${ITEM}"
sleep 10

$ cat samples.txt
alpha
bravo
charlie
delta
echo

$ cat slurm-34396612_*
task_id=1 item=alpha
task_id=3 item=charlie
task_id=5 item=echo
```

Job Array - A More Realistic Example

including populating environment variables

```
$ cat more_samples.csv
sampleA,0.10,128,true
sampleB,0.20,256,false
sampleC,0.05,064,true
sampleD,0.50,032,false
sampleE,0.75,512,true

$ cat process.sh
#!/bin/bash

ROW="$(sed -n "${SLURM_ARRAY_TASK_ID}p" more_samples.csv)"
#parse simple CSV into fields.
IFS=',' read -r ID THRESHOLD STEPS DRYRUN <<<"$ROW"
#Signifiy the delimiter, put the values into the bash shell $ENV

#see how it is interpreted as new bash variables
echo "task=${SLURM_ARRAY_TASK_ID} id=${ ID} thr=${THRESHOLD} steps=${STEPS} dry=${DRYRUN}"

#run one command with one set of parameters, repeated n times for each --array value.
python3 my_python_workflow.py \
  --id "${ID}" \
  --threshold "${ THRESHOLD}" \
  --steps "${ STEPS}" \
  --dry-run "${ DRYRUN}"
```

Job Array - Clean Submission Indices

Show the user how easy it is to make sense of their job queue.

Rather than job IDs just being an arbitrary number, it is now meaningfully describing what trial / parameter is being evaluated!

```
$ sbatch --wrap="sleep 10" --array=1-50 -c 1 -p general -q normal
$ squeue
             JOBID       PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
         15484570_[5-50]   general     wrap    rocky PD       0:00      1 (Resources)
              15484570_2   general     wrap    rocky  R       0:02      1 lci-compute-01-1
              15484570_3   general     wrap    rocky  R       0:02      1 lci-compute-01-2
              10548457_4   general     wrap    rocky  R       0:02      1 lci-compute-01-2
              10548457_1   general     wrap    rocky  R       0:03      1 lci-compute-01-1
```

Have multiple, unrelated jobs? Now their individual progress is logically connected:

```
$ sbatch --wrap="sleep 10" --array=1-50 -c 1 -p general -q normal
$ squeue
             JOBID       PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
         15484571_[1-50]   general     wrap    rocky PD       0:00      1 (Resources)
         15484570_[5-50]   general     wrap    rocky PD       0:00      1 (Resources)
```

Array Size Maximum

Array sizes are governed by your `slurm.conf` and can be viewed with `scontrol`:

```
$ grep Array /etc/slurm/slurm.conf
MaxArraySize=200000

$ scontrol show config | grep Array
MaxArraySize            = 200000
```

Changes to `slurm.conf` require `scontrol reconfig` to take effect.

Array Indexing / Bulk Commands

You can address multiple subjobs at once with commas:

```
# submit an array job
$ sbatch --array=1-300 noop.sh
Submitted batch job 878300

# update a subset of array jobs
$ scontrol update job=878300_1 timelimit=0-4
$ scontrol update job=878300_[5,30] timelimit=0-4
$ scontrol update job=878300_[111,201] timelimit=0-4
$ scontrol update job=878300_[1,191,202] timelimit=0-8

# cancel a subset of array jobs
$ scancel 878300_1
$ scancel 878300_[5,30]
$ scancel 878300_[111,201]

# update all (remaining) jobs in an array
$ scontrol update job 878300 timelimit=1-0

# cancel all (remaining) jobs in an array
$ scancel 878300
```

Array Indexing / Bulk Commands

You can also address multiple subjobs at once with range specifiers: -

```
# submit an array job
$ sbatch --array=1-300 noop.sh
Submitted batch job 878300

# update a subset of array jobs
$ scontrol update job=878300_1 timelimit=0-4
$ scontrol update job=878300_[5-30] timelimit=0-4
$ scontrol update job=878300_[111-115] timelimit=0-4
$ scontrol update job=878300_[31-110,116-201] timelimit=0-8

# cancel a subset of array jobs
$ scancel 878300_1
$ scancel 878300_[5-30]
$ scancel 878300_[116-201]

# update all (remaining) jobs in an array
$ scontrol update job 878300 timelimit=1-0

# cancel all (remaining) jobs in an array
$ scancel 878300
```

Array Throttling

You can limit maximum concurrency of your job arrays with `%`:

 # submit an array job, maximum concurrent running jobs at 5 simultaneously

```
$ sbatch --array=1-300%5 noop.sh
```

User jobs should have all the information/data they need to finish a job once allocated, so user-driven throttling cases are not super common, though certainly some cases will exist.

However, infrastructure may not be able to keep up with some cases, and admins might recommend/require throttling to sidestep some submissions that cause I/O bottlenecks or RPC overloads.

Array Throttling (cont.)

Example:

```
# sbatch –array=1-200000 -c 1 -t 30 process_pdf_file.py
```

The slurm configuration might permit 200k subjobs, and the cluster may have tens of thousands of CPUs available to service this array, but thousands or tens of thousands of jobs might make too many **simultaneous** `slurmctld` RPC calls overwhelming Slurm's 1000 RPC call limit.

This results in jobs failing to get cleaned up and usually manifests as nodes being left in the COMPLETING state, potentially requiring admin/user intervention.

Throttling to a few hundred will permit an acceptable throughput without causing too much `slurmctld` overheads and hangups.

Insufficient MaxJobCount

Slurm does *not* give a great user experience if the amount of jobs a user submits exceeds the permitted MaxJobCount.

In `slurm.conf`:

```
[root@lci-head-02-1 ~]# scontrol show config | grep MaxArray
MaxArraySize            = 300
[root@lci-head-02-1 ~]# scontrol show config | grep MaxJobCount
MaxJobCount             = 50
```

For the user:

```
[rocky@lci-head-02-1 slurm]$ sbatch --array=1-200 --wrap="sleep 5" -p general
sbatch: error: Slurm temporarily unable to accept job, sleeping and retrying
```

In `slurmctld.log`:

```
[2025-10-18T22:49:03.219] error: job_allocate: MaxJobCount limit from slurm.conf
reached (50)
[2025-10-18T22:49:03.219] _slurm_rpc_submit_batch_job: Resource temporarily unavailable
```

Job Arrays

Thank you!

Any questions?